

Capítulo 01

Bigram Language Model

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 30/07/2026

Capítulo 01 — Bigram Language Model

Objetivo de aprendizagem: entender o que significa "prever o próximo token" e treinar o modelo de linguagem mais simples possível — primeiro **por contagem**, depois como uma **rede neural** — e perceber que as duas abordagens são, no fundo, a mesma coisa.

Pré-requisitos deste capítulo: Python básico e a noção de que uma matriz é uma tabela de números. Nada além disso. Tudo o que for de probabilidade ou cálculo é introduzido aqui.

Arquivos: - `bigram.py` — versão por contagem (*count-based*) - `bigram_nn.py` — versão rede neural (*neural network*) - `names.txt` — dataset de nomes - `exercicios.md` — exercícios

1. O que é um modelo de linguagem?

Um **modelo de linguagem** (*language model*) é, essencialmente, uma máquina que responde a uma pergunta:

Dado o que já vi, o que vem em seguida?

Quando o seu celular sugere a próxima palavra enquanto você digita, é um modelo de linguagem trabalhando. Quando o ChatGPT escreve um texto, é o mesmo princípio, levado ao extremo: ele gera **um token de cada vez**, sempre perguntando "qual o próximo token mais provável, dado tudo que veio antes?".

Esse é o conceito central de todo o curso. Um LLM gigante e um modelo de brinquedo de uma linha fazem **a mesma tarefa** — prever o próximo token. A diferença está em quão bem eles fazem isso. Vamos começar pelo modelo de brinquedo.

Token? Caractere?

Um **token** é a "unidade" que o modelo manipula. Em modelos de verdade, um token costuma ser um pedaço de palavra (vamos construir um tokenizador no Capítulo 6). Aqui, para simplificar, **cada token é um único caractere** (uma letra). É o famoso *character-level language model*.

Nossa tarefa concreta: inventar nomes

Em vez de "escrever textos", vamos resolver um problema pequeno e divertido: **gerar nomes de pessoas que soam plausíveis**, mas que não existem necessariamente.

Para isso temos o arquivo `names.txt`, com um nome por linha:

```
ana
maria
joao
jose
pedro
...
```

O modelo vai olhar esses nomes reais e aprender os "padrões de letras" do português — que nomes costumam começar com certas letras, que **q** quase sempre é seguido de **u**, que muitos nomes terminam em **a** ou **o**, etc. Depois, ele inventa nomes novos seguindo esses padrões.

2. O modelo mais simples: o bigrama

A palavra **bigrama** (*bigram*) significa "dois elementos seguidos". Um modelo de bigrama faz a aposta mais ingênua possível:

Para prever o próximo caractere, **só olho o caractere atual**. Ignoro todo o resto.

É pouco contexto (no nome **maria**, ao prever a letra depois do segundo **a**, o modelo nem lembra que tinha um **m** no começo). Mas é o suficiente para aprender coisas úteis e é o ponto de partida perfeito.

Onde começa e onde termina um nome?

Um detalhe importante: o modelo precisa saber **quando começar** (qual a primeira letra provável de um nome?) e **quando parar** (a palavra acabou?). Resolvemos isso com um **token especial de fronteira**, que representamos pelo ponto **.**:

- Todo nome é tratado como **.** + nome + **.**
- O nome **ana** vira a sequência: **. a n a .**

Assim, o par **(. , a)** ensina "nomes podem começar com **a**", e o par **(a , .)** ensina "depois de **a** o nome pode acabar".

Listando todos os bigramas de **ana**:

```
.a (começo -> a)
an (a -> n)
na (n -> a)
a. (a -> fim)
```

3. Construindo o vocabulário

Antes de contar, precisamos transformar caracteres em números (computadores trabalham com números, não com letras). Criamos dois dicionários:

- **stoi** (*string to integer*): mapeia cada caractere para um índice.
- **itos** (*integer to string*): o caminho de volta.

```
chars = sorted(list(set("".join(words)))) # ['a', 'b', 'c', ..., 'z']
stoi = {c: i + 1 for i, c in enumerate(chars)}
stoi["."] = 0 # o token de fronteira fica no índice 0
itos = {i: c for c, i in stoi.items()}
vocab_size = len(itos) # 27 = 26 letras + '.'
```

Nosso **vocabulário** tem 27 tokens: as 26 letras de **a** a **z** mais o **.**.

4. Aprendendo por contagem

A ideia é direta: vamos **percorrer todos os nomes e contar**. Quantas vezes a letra **a** é seguida de **n**? Quantas vezes **q** é seguido de **u**? E assim por diante.

Guardamos tudo numa **matriz N** de tamanho 27×27. A célula **N[i, j]** é "quantas vezes o token **j** apareceu logo depois do token **i**".

```
N = torch.zeros((vocab_size, vocab_size), dtype=torch.int32)
for w in words:
    chs = ["."] + list(w) + ["."]
    for ch1, ch2 in zip(chs, chs[1:]): # cada par de caracteres vizinhos
        N[stoi[ch1], stoi[ch2]] += 1
```

zip(chs, chs[1:]) é um truque comum em Python para percorrer pares vizinhos. Se **chs = ['.', 'a', 'n', 'a', '.']**, então **zip** produz **('.', 'a'), ('a', 'n'), ('n', 'a'), ('a', '.')**. Exatamente nossos bigramas.

Ao final, a **linha i** da matriz **N** é o "perfil" do token **i**: ela diz, dentre todas as vezes que **i** apareceu, o que costumou vir depois.

De contagens para probabilidades

Contagens são números absolutos ("o **a** foi seguido de **n** umas 120 vezes"). Para gerar nomes, queremos **probabilidades** ("dado que estou no **a**, qual a chance de o próximo ser **n**?"). Para isso, **normalizamos cada linha**: dividimos cada valor pela soma da linha, de modo que cada linha some 1.0.

```
P = (N + 1).float()
P = P / P.sum(dim=1, keepdim=True) # cada linha agora soma 1.0
```

Agora **P[i]** é uma **distribuição de probabilidade**: 27 números entre 0 e 1 que somam 1, dizendo a probabilidade de cada token vir depois de **i**.

Probabilidade em uma frase: é só um número entre 0 (impossível) e 1 (certo) que mede a chance de algo acontecer. Se um dado honesto tem 6 faces, a probabilidade de cada face é $1/6 \approx 0,167$. Aqui, em vez de 6 faces, temos 27 "faces" (os tokens), e o dado é **viciado** de um jeito diferente para cada caractere atual.

Por que o + 1? (model smoothing)

Repare no **(N + 1)**. Alguns bigramas **nunca aparecem** no treino — talvez nenhum nome tenha **j** depois de **q**. A contagem seria 0, a probabilidade seria 0, e — como veremos na avaliação — **log(0)** é **-infinito**, o que quebra as contas. Somar 1 a todas as contagens garante que **nada** tenha probabilidade exatamente zero. Isso se chama **suavização de Laplace** (*Laplace / add-one smoothing*). Quanto maior o número somado, mais "uniforme" (chutômetro) o modelo fica.

5. Gerando nomes (sampling)

Temos a tabela **P**. Como inventar um nome? **Amostrando** (*sampling*), caractere a caractere:

1. Comece no token de fronteira **.** (índice 0).
2. Olhe a linha **P[ix]** — a distribuição do próximo caractere.
3. **Sorteie** um caractere segundo essa distribuição.
4. Se sorteceu o **.**, o nome acabou. Senão, anote a letra e volte ao passo 2 com ela.

```
g = torch.Generator().manual_seed(2147483647) # semente fixa -> resultado reprodutível

def sample_name():
    out = []
    ix = 0
    while True:
        p = P[ix]
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        if ix == 0:
            break
        out.append(itos[ix])
    return "".join(out)
```

torch.multinomial(p, ...) é o "sorteio viciado": dada a distribuição **p**, ele devolve um índice escolhido com aquela probabilidade. Tokens mais prováveis são sorteados mais vezes, mas os improváveis ainda têm chance — é isso que dá variedade aos nomes gerados.

manual_seed fixa o gerador de números aleatórios para que você obtenha sempre os mesmos resultados. Útil para estudar e depurar; em produção você deixaria aleatório de verdade.

Rode **python bigram.py**. Os nomes saem meio estranhos (**mor**, **aya**, **kenadi**...) porque o bigrama tem memória curtíssima — mas já são bem mais "pronunciáveis" do que letras totalmente aleatórias. O modelo aprendeu **alguma** estrutura do português.

6. O modelo é bom? (a loss)

"Os nomes parecem ok" não é uma medida científica. Precisamos de **um número** que diga o quão bom o modelo é — de preferência, um número que possamos tentar **minimizar**. Esse número é a *loss* (função de perda/custo).

A ideia: um bom modelo atribui **alta probabilidade** aos bigramas que realmente acontecem nos dados reais. Então vamos medir, para cada bigrama do dataset, qual probabilidade o modelo deu a ele, e combinar tudo num só número.

Likelihood, log-likelihood e NLL

- A **likelihood** (verossimilhança) é o produto das probabilidades de todos os bigramas. Problema: multiplicar milhares de números entre 0 e 1 dá um valor minúsculo, que o computador arredonda para

zero.

- Por isso usamos o **logaritmo**. Uma propriedade-chave: $\log(a \cdot b) = \log(a) + \log(b)$. Logo, em vez de **multiplicar** as probabilidades, podemos **somar** os seus logaritmos. Isso é a **log-likelihood**, numericamente estável.
- O log de um número entre 0 e 1 é **negativo** (ex.: $\log(0,1) \approx -2,3$). Para trabalharmos com um número **positivo que queremos minimizar**, pegamos o **negativo** e tiramos a **média**. Isso é a **negative log-likelihood (NLL)** — a nossa *loss*.

```
log_likelihood = 0.0
n = 0
for w in words:
    chs = ["."] + list(w) + ["."]
    for ch1, ch2 in zip(chs, chs[1:]):
        prob = P[stoi[ch1], stoi[ch2]]
        log_likelihood += torch.log(prob)
    n += 1
nll = -log_likelihood / n
```

Interpretação: quanto **menor** a NLL, melhor o modelo. O melhor valor possível seria 0 (o modelo acerta tudo com probabilidade 1 — impossível na prática). Um modelo que chuta uniformemente entre 27 tokens teria $\text{loss} \approx \log(27) \approx 3,30$. Nosso bigrama fica bem abaixo disso: ele realmente aprendeu algo.

Guarde esse valor de loss. No próximo passo, vamos treinar uma rede neural e ver que ela aprende **essencialmente a mesma tabela** — chegando a uma loss da mesma ordem (e idêntica, quando igualamos a suavização dos dois lados, como você verá no exercício E5).

7. A virada de chave: o mesmo modelo como rede neural

Até aqui, "treinar" foi só contar. Agora vem a ideia que sustenta o resto do curso: vamos obter **as mesmas probabilidades** sem contar — deixando uma **rede neural aprendê-las por otimização**. Parece um retrocesso (por que complicar?), mas é o contrário: contar só funciona para o bigrama. A abordagem de otimização vai escalar até o Transformer. Aqui a gente a aprende no caso mais simples possível.

One-hot: transformando um índice num vetor

A rede não recebe o índice **5** direto; ela recebe um **vetor one-hot**: um vetor de 27 posições, todo zero, com um único **1** na posição do caractere. O caractere de índice 5 vira $[0, 0, 0, 0, 0, 1, 0, \dots, 0]$.

```
xenc = F.one_hot(xs, num_classes=vocab_size).float() # (num_exemplos, 27)
```

A "rede": uma única matriz de pesos W

Nosso modelo é uma só camada linear, representada por uma matriz de pesos W de 27×27 , com valores **aleatórios** no início:

```
W = torch.randn((vocab_size, vocab_size), generator=g, requires_grad=True)
```

O cálculo central é uma **multiplicação de matrizes** (*matmul*): `xenc @ W`.

Por que isso "seleciona uma linha"? Multiplicar um vetor one-hot (com o **1** na posição **i**) por **W** resulta exatamente na **linha i de W**. Ou seja: cada linha de **W** desempenha o papel que a linha de **N** tinha — ela guarda os "números brutos" do que vem depois do token **i**. A diferença é que esses números agora serão **aprendidos**, não contados.

De números brutos a probabilidades: o softmax

A saída de `xenc @ W` são os **logits** — números reais quaisquer (podem ser negativos). Precisamos convertê-los em probabilidades (positivas, somando 1). Fazemos isso em dois passos, que juntos se chamam **softmax**:

```
logits = xenc @ W # números reais quaisquer (log-counts)
counts = logits.exp() # exp() deixa tudo positivo (equivale a N)
probs = counts / counts.sum(dim=1, keepdim=True) # normaliza -> soma 1
```

Repare na simetria com a versão por contagem: lá tínhamos **N** (positivo) e normalizávamos. Aqui, `logits.exp()` faz o papel de **N** (o `exp` garante valores positivos), e normalizamos igual. Por isso os logits são chamados de **log-counts**: exponenciados, viram contagens.

8. Como a rede aprende: gradient descent

No começo **W** é aleatório, então as probabilidades são lixo e a loss é alta. Treinar é **ajustar W aos poucos para baixar a loss**. O algoritmo é o *gradient descent* (descida do gradiente), e tem três passos repetidos muitas vezes:

```
for step in range(200):
    # 1) FORWARD: calcula as probabilidades e a loss a partir do W atual
    logits = xenc @ W
    counts = logits.exp()
    probs = counts / counts.sum(dim=1, keepdim=True)
    loss = -probs[torch.arange(num), ys].log().mean() + 0.01 * (W ** 2).mean()

    # 2) BACKWARD: descobre, para cada peso, se aumentá-lo sobe ou desce a loss
    W.grad = None
    loss.backward()

    # 3) UPDATE: empurra cada peso na direção que DIMINUI a loss
    W.data += -50 * W.grad
```

Vamos destrinchar cada passo.

Forward. Note `probs[torch.arange(num), ys]`: para cada exemplo, isso pega a probabilidade que o modelo atribuiu ao caractere **correto** (**ys** é o alvo). É exatamente a mesma NLL da Seção 6 — só que calculada de uma vez para todos os exemplos. O termo extra `0.01 * (W**2).mean()` é a

regularização (próxima seção).

Backward — o gradiente. Aqui está a mágica. O **gradiente** é a derivada da loss em relação a cada peso: um número que diz "se eu aumentar um tiquinho este peso, a loss sobe ou desce, e com que força?". Calcular isso à mão para milhares de pesos seria inviável; o PyTorch faz **automaticamente** com `loss.backward()`, preenchendo `W.grad`. Esse mecanismo se chama **autograd / backpropagation** — e é tão central que o Capítulo 2 inteiro é dedicado a construí-lo do zero. Por ora, confie: depois do `backward()`, `W.grad` diz a "inclinação" da loss em cada direção.

`W.grad = None` antes de cada `backward()` zera os gradientes. Sem isso, o PyTorch **acumula** os gradientes de uma iteração na outra, e o treino dá errado.

Update. Para **descer** a loss, andamos na direção **oposta** ao gradiente — por isso o sinal de menos. O `50` é a *learning rate* (taxa de aprendizado): o tamanho do passo. Grande demais, o treino fica instável; pequeno demais, fica lento. (Ajustar a learning rate direito é tema do Capítulo 7.)

Rode `python bigram_nn.py` e veja a loss **caindo** a cada passo, estabilizando em torno de **~2,2**. A versão por contagem deu **~2,4**. "Perto, mas não igual" — e a explicação é exata e importante:

Por baixo do pano, os dois modelos buscam **a mesma tabela ótima** (o *MLE* — *maximum likelihood estimate*), que neste dataset tem loss $\approx 2,14$. A diferença que você vê entre os dois arquivos vem **só do quanto cada um foi suavizado**: o `+1` da contagem puxa mais forte para o uniforme do que o `0.01` da rede. Iguale os dois botões e as losses se encontram (faça isso no exercício **E5**).

Ou seja, **suavização (+1) e regularização (L2) são o mesmo botão**, vestido de duas formas. Tirando esse botão, contagem e rede produzem o mesmo modelo.

Esse é o recado do capítulo, em uma frase:

Contar é um atalho que só funciona no bigrama. Otimizar uma rede por gradient descent é o método geral — e é o que vamos escalar até o Transformer.

9. Regularização = smoothing, de novo

Lembra do `+1` que evitava probabilidade zero na versão por contagem? A rede tem um equivalente: o termo `0.01 * (W**2).mean()` na loss. Ele **penaliza pesos grandes**, empurrando `W` na direção do zero. E quando todos os pesos são iguais (zero), todos os logits são iguais, e o softmax dá uma distribuição **uniforme** — o máximo de suavização. Aumente esse `0.01` e os nomes gerados ficam mais "genéricos"; diminua e o modelo gruda mais nos dados. É o mesmo trade-off do `+1`, com outra roupa. Isso se chama **L2 regularization** (ou *weight decay*).

10. Resumo do capítulo

- Um **modelo de linguagem** prevê o próximo token. Tudo no curso é uma variação disso, em escala crescente.
- O **bigrama** prevê olhando só o caractere atual. Treinamos de duas formas:

- **por contagem**: monta a matriz **N**, normaliza para **P**;
- **por rede neural**: uma matriz **W**, softmax, e gradient descent.
- As duas chegam à **mesma loss** — contar é um caso particular de otimizar.
- Conceitos que vão reaparecer o curso inteiro: **token**, **vocabulário**, **one-hot**, **logits**, **softmax**, **loss (NLL)**, **gradient descent**, **backpropagation**, **learning rate**, **regularização/smoothing**, **sampling**.

O que vem no Capítulo 2

Neste capítulo, `loss.backward()` foi uma caixa-preta mágica. No **Capítulo 02 — Micrograd**, a gente abre a caixa: vamos construir um motor de **autograd** do zero, entendendo exatamente como o gradiente de cada peso é calculado pela **backpropagation**. É a peça que faz toda rede neural — do bigrama ao GPT-4 — conseguir aprender.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 01 (Bigram Language Model)

Faça na ordem; os primeiros são de fixação, os últimos são mais desafiadores. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

E1 — Leitura de código (aquecimento)

Sem rodar, responda: 1. Por que `vocab_size` é 27, e não 26? 2. No `bigram.py`, o que aconteceria se trocássemos `(N + 1)` por `N` (sem suavização) ao calcular a loss? Por quê? 3. Para que serve o `manual_seed`?

E2 — Mexa nos dados

Adicione 5 nomes ao final de `names.txt` (pode ser de amigos, ou nomes inventados). Rode `bigram.py` de novo. A loss subiu ou desceu? Os nomes gerados mudaram? Explique em 1-2 frases por que adicionar dados muda o modelo.

E3 — O efeito da suavização (*smoothing*)

No `bigram.py`, troque o `+ 1` por `+ 0`, `+ 10` e `+ 100` (três execuções). 1. Como a **loss** muda em cada caso? 2. Como os **nomes gerados** mudam (mais "criativos" ou mais "genéricos")? 3. Explique: por que `+ 0` pode dar problema?

E4 — Temperatura no sampling

No `bigram.py`, antes de amostrar, eleve a distribuição a uma potência e re-normalize: `p = P[ix] ** T; p = p / p.sum()`. Teste `T = 0.3`, `T = 1.0` e `T = 3.0`. 1. Com `T` baixo, os nomes ficam mais "óbvios" ou mais "aleatórios"? E com `T` alto? 2. Relacione isso com o conceito de **temperatura** que você já ouviu falar em LLMs.

E5 — Faça contagem e rede convergirem (a equivalência)

Rodando os dois arquivos como vêm, você acha a loss por contagem (~2,4) maior que a da rede (~2,2). Elas *deveriam* ser a mesma coisa — vamos provar igualando a "quantidade de suavização" dos dois lados: 1. No `bigram.py`, troque o `+ 1` por `+ 0.01`. Qual a nova loss por contagem? (Deve cair para ~2,15 — perto do MLE, o melhor possível.) 2. No `bigram_nn.py`, zere a regularização (troque `0.01 *` por `0.0 *`) e aumente os passos de 200 para 1000. Qual a loss pura da rede agora? 3. Os dois valores se encontram em torno de ~2,1? Explique, com suas palavras, por que `+1` (smoothing) e o termo `(W**2)` (regularização L2) são "o mesmo botão".

E6 — Avaliando um nome específico (desafio)

Escreva uma função `nll_de(nome)` que calcule a negative log-likelihood **de um único nome** segundo o modelo de contagem `P`. Use-a para responder: 1. Qual tem menor NLL (é mais "provável" segundo o modelo): `ana` ou `xwq`? 2. Teste com o seu próprio nome. Ele é "provável" para o modelo?

E7 — Rumo ao trigrama (desafio, antecipa o Cap. 3)

Modifique o modelo de contagem para um **trigrama**: prever o próximo caractere a partir dos **dois** caracteres anteriores. 1. Em vez de uma matriz 27×27 , qual o formato da tabela de contagens agora? 2. A loss melhora em relação ao bigrama? Por quê? 3. Que novo problema de "dados esparsos" (muitos zeros) começa a aparecer? Como a suavização ajuda — e por que ela não resolve tudo? (Essa limitação é exatamente o que motiva o MLP do Capítulo 3.)